Testing a React App

> Part of the React curriculum. This document covers Vitest/Jest, React Testing Library, mocking, testing custom hooks, testing the view-model/URL-state/section patterns of a data-heavy dashboard, and where end-to-end testing fits.

1. Vitest / Jest + React Testing Library

React Testing Library encourages testing components the way a user would interact with them — querying rendered output by visible text or accessibility role, rather than by internal implementation details:

```
import { render, screen } from '@testing-library/react';
import { vi } from 'vitest';
import { api } from '../api/client';
import OverviewPage from './OverviewPage';

vi.mock('../api/client', () => ({ api: { overview: vi.fn() } }));

test('renders KPIs from the API response', async () => {
  vi.mocked(api.overview).mockResolvedValue({ total: 1234, pct_change: 12.3, breakdown: [] });
  render(<OverviewPage />);
  expect(await screen.findByText('1,234')).toBeInTheDocument();
});

test('shows an error message instead of crashing on API failure', async () => {
  vi.mocked(api.overview).mockRejectedValue(new Error('Failed to load data.'));
  render(<OverviewPage />);
  expect(await screen.findByText('Failed to load data.')).toBeInTheDocument();
});
```

`screen.findByText(...)` is asynchronous (it waits/retries until the text appears, or times out) — appropriate here since the data arrives after an awaited API call; `screen.getByText` is the synchronous equivalent, appropriate when you don't need to wait for anything. Querying by role (`screen.getByRole('button', { name: 'Submit' })`) is generally preferred over querying by CSS class or test ID where possible, since it also serves as a lightweight accessibility check — if you can't find an element by its accessible role/name, a real assistive-technology user likely couldn't either.

1.1 User interaction

`@testing-library/user-event` simulates real user interactions more faithfully than directly firing DOM events:

```
import userEvent from '@testing-library/user-event';

test('clicking submit calls the handler', async () => {
  const user = userEvent.setup();
  const handleSubmit = vi.fn();
  render(<Form onSubmit={handleSubmit} />);

  await user.type(screen.getByLabelText('Name'), 'Alice');
  await user.click(screen.getByRole('button', { name: 'Submit' }));

  expect(handleSubmit).toHaveBeenCalledWith('Alice');
});
```

2. Mock the API client — never hit a live backend in a component test

Component tests should be fast, deterministic, and independent of whether a backend happens to be running. `vi.mock(...)` replaces the API client module entirely within a test file, so you control exactly what each call to it returns (a successful response, an error, an empty result) and can assert the component's loading/error/data behavior (Core Concepts, §5) directly.

This also means the API client module's *own* logic (URL/query-string construction, error mapping) needs its own separate, dedicated test that mocks the lower-level `fetch` call instead, since mocking the client itself would skip over exactly the code that test is meant to check:

```ts
import { vi } from 'vitest';
import { api } from './client';

function mockFetchOnce(body: unknown, init: { ok?: boolean; status?: number } = {}) {
  const { ok = true, status = 200 } = init;
  vi.stubGlobal('fetch', vi.fn().mockResolvedValue({
    ok, status, json: () => Promise.resolve(body),
  }));
}

test('overview() fetches the right URL', async () => {
  mockFetchOnce({ total: 1234 });
  await api.overview();
  expect(fetch).toHaveBeenCalledWith('/api/overview');
});

test('throws ApiError with the response detail on a non-ok response', async () => {
  mockFetchOnce({ detail: 'not found' }, { ok: false, status: 404 });
  await expect(api.overview()).rejects.toMatchObject({ status: 404, message: 'not found' });
});
```

3. Testing custom hooks

A custom hook can't be called directly outside a component (hooks rely on React's rendering machinery), but `@testing-library/react`'s `renderHook` provides a minimal harness for testing one in isolation:

```ts
import { renderHook, waitFor } from '@testing-library/react';
import { useAsync } from './useAsync';

test('useAsync reflects the loading, then success, states', async () => {
  const { result } = renderHook(() => useAsync(() => Promise.resolve({ value: 42 }), []));

  expect(result.current.loading).toBe(true);

  await waitFor(() => expect(result.current.loading).toBe(false));
  expect(result.current.data).toEqual({ value: 42 });
});
```

`result.current` always reflects the hook's most recent return value; `waitFor` polls until the given assertion passes (or times out), which is necessary here since the hook's state changes asynchronously after the mocked promise resolves.

4. Stub heavy third-party rendering

If a component renders something with real, complex DOM/rendering behavior that the test environment doesn't fully support — a canvas-based chart library is a common example, since most component tests run in a simulated DOM (like jsdom), not a real browser, and things like real canvas painting or animation-frame timing often don't behave the same way there — replace it with a trivial stub in the test and assert only on the *props* passed to it:

```
vi.mock('some-chart-library', () => ({
  Chart: (props: { ariaLabel?: string }) => <div data-testid="chart" aria-label={props.ariaLabel}
/>,
}));
```

This keeps the responsibility boundary clean: a page-level test verifies *this component* wired the right data into the chart; it does not (and should not try to) verify the chart library's own rendering correctness — that's a separate concern, ideally covered by that library's own test suite (or, if it's a shared internal component library, that library's own tests, per Routing & API Integration, §4).

5. Where end-to-end testing fits

Everything above runs in a simulated browser environment (jsdom), testing components in isolation with mocked network calls. **End-to-end (E2E) testing** tools — Playwright and Cypress are the two most common — drive a *real* browser against a *real*, fully running application (frontend and backend both actually running, usually against test/staging data), clicking through complete user flows exactly as a real user would.

E2E tests are slower and more expensive to run and maintain than component tests, so they're typically reserved for a small number of the most critical, highest-value user flows (e.g., "can a user load the app and see the main dashboard page render with data") — not a replacement for component-level tests, but a complement covering what component tests fundamentally can't: whether the frontend and a real backend actually work together correctly end to end.

6. Testing the patterns of a data-heavy dashboard

Patterns for Data-Heavy Dashboards moves most logic out of components, which makes most of it cheap to test:

- **Builders (view-model functions) are tested as plain functions** — no rendering. Use fixtures that include the awkward cases: a gap in the middle of a series, a series that ends early, an entity with no values at all, a year the response doesn't contain. Assert that missing values are *omitted*, not zero.
- **Fixtures use the API's exact wording.** If a licence, a caveat or a source name appears in the UI, copy the real string from the API response into the fixture. A fixture that paraphrases it lets a test pass while the real text would not match (and lets a copy-edit on the API side break the UI unnoticed).
- **Independent sections:** make *one* request reject and assert the other sections still render and the failing one shows its message — the property the architecture exists to provide.

- **URL-backed state:** render the hook inside a router with a controlled initial URL; assert that an unknown value falls back, that setting the default removes the parameter, that `replace` is used, and that the hash and other parameters survive.
- **Phone variants:** stub `matchMedia` and assert only one of the two controls is in the DOM.
- **Time and animation:** use fake timers; assert the animation stops at the end of the range and on unmount.
- **Contracts shared with other code** (route paths, anchor ids, query-parameter names that an agent links to): export them from one module, and let the other side's test read them. Stub only the heavy renderer (`SyChart`); a page smoke test per page (loading → data → error) is the cheapest regression net.

7. Core libraries summary

Library	Used for
Vitest (or Jest)	the test runner
React Testing Library	rendering components and querying output the way a user would
`@testing-library/user-event`	simulating realistic user interactions
Playwright / **Cypress**	end-to-end testing against a real running application

See also

- React curriculum index
- Core Concepts — the components and hooks under test
- Deployment — running the tested application in production